

Table 17. CPR and CMD scores for Llama-3.1 on the arithmetic public test sets, separated by operator. Scores are generally similar across operators, and methods follow similar rankings regardless of which operators are used. A notable exception is EAP-IG-activations, where CPR scores are significantly different.

Method	Arithmetic (+)		Arithmetic (−)	
	CPR (↑)	CMD (↓)	CPR (↑)	CMD (↓)
Random	0.25	0.25	0.75	0.75
EAP (mean)	0.40	0.07	0.31	0.07
EAP (CF)	0.49	0.01	0.61	0.01
EAP-IG-inputs	0.96	0.00	1.03	0.00
EAP-IG-activations	0.98	0.00	0.99	0.00
NAP (CF)	0.26	0.29	0.27	0.27
NAP-IG (CF)	0.43	0.18	0.34	0.18
IFR	0.90	0.24	0.87	0.20

F. Details on Causal Variable Localization Track

F.1. Causal Abstraction Analysis

Causal Models and Interventions A deterministic causal model $\mathcal{H}$ has *variables* that take on *values*. Each variable has a *mechanism* that determines the value of the variable based on the values of *parent variables*. Variables without parents, denoted $\mathbf{X}$, can be thought of as inputs that determine the setting of all other variables, denoted $\mathcal{H}(\mathbf{x})$. A *hard intervention* $X \leftarrow x$ overrides the mechanisms of variable X , fixing it to a constant value x .

Interchange Interventions We perform *interchange interventions* (Vig et al., 2020; Geiger et al., 2020) where a variable (or set of features) X is fixed to be the value it would take on if the LM were processing *counterfactual input* c . We write $X \leftarrow \text{Get}(\mathcal{H}(c), X)$ where $\text{Get}(\mathcal{H}(c), X)$ is the value of variable X when $\mathcal{H}$ processes input c . In experiments, we will feed a *base input* b to a model under an interchange intervention $\mathcal{H}_{X \leftarrow \text{Get}(\mathcal{H}(c), X)}(b)$.

Featurizing Hidden Vectors The dimensions of hidden vectors are not an ideal unit of analysis (Smolensky, 1986), and so it is typical to *featurize* a hidden vector using some invertible function, e.g., an orthogonal matrix, to project a hidden vector into a new variable space with more interpretable dimensions called “features” (Geiger et al., 2024a; Huang et al., 2024a). A feature intervention $\Pi \leftarrow \Pi$ edits the mechanism of a hidden vector $\mathbf{h}$ to fix the value of features Π to Π .

Alignment The LM is a *low-level causal model* $\mathcal{N}$ where variables are dimensions of hidden vectors and the hypothesis about LM structure is a *high-level causal model* $\mathcal{H}$. An *alignment* assigns each high-level variable X to features of a hidden vector $\Pi_{\mathbf{h}}^X$, e.g., orthogonal directions in the activation space of $\mathbf{h}$. To evaluate an alignment, we perform intervention experiments to evaluate whether high-level interventions on the variables in $\mathcal{H}$ have the same effect as interventions on the aligned features in $\mathcal{N}$.

Causal Abstraction We use interchange interventions to reveal whether the hypothesized causal model $\mathcal{H}$ is an abstraction of an LM $\mathcal{N}$. To simplify, assume both models share an input and output space. The high-level model $\mathcal{H}$ is an abstraction of the low-level model $\mathcal{N}$ under a given alignment when each high-level interchange intervention and the aligned low-level intervention result in the same output. For a high-level intervention on X aligned with low-level features $\Pi_{\mathbf{h}}^X$ with a counterfactual input c and base input b , we write

$$\text{GetOutput}(\mathcal{N}_{\Pi_{\mathbf{h}}^X \leftarrow \text{Get}(\mathcal{N}(c), \Pi_{\mathbf{h}}^X)}(b)) = \text{GetOutput}(\mathcal{H}_{X \leftarrow \text{Get}(\mathcal{H}(c), X)}(b)) \quad (7)$$

If the low-level interchange intervention on the LM produces the same output as the aligned high-level intervention on the algorithm, this is a piece of evidence in favor of the hypothesis. This extends naturally to multi-variable interventions (Geiger et al., 2024a).

Graded Faithfulness Metric We construct *counterfactual datasets* for each causal variable where an example consists of a base prompt and a counterfactual prompt. The *counterfactual label* is the expected output of the algorithm after the high-level interchange intervention, i.e., the right-side of Equation 7. The interchange intervention accuracy is the proportion of examples for which Equation 7 holds, i.e., the degree to which $\mathcal{H}$ faithfully abstracts $\mathcal{N}$.

F.2. Aligning Unsupervised Features to Causal Variables

In our experiments, we use a variety of unsupervised methods for featurizing hidden vectors in LMs, including principal component analysis (PCA), sparse autoencoders (SAE), and simply taking standard dimensions of the hidden vector as features. For a variable X in the high-level causal model $\mathcal{H}$, we learn a set of features $\Pi_{\mathbf{h}}^X$ of a hidden vector $\mathbf{h}$ of the LM $\mathcal{N}$ using Differential Binary Masking (DBM) (Cao et al., 2020; 2022; Csordás et al., 2021; Davies et al., 2023). Given base input b and counterfactual input c , we train a mask $\mathbf{m} \in [0, 1]^{|\Pi_{\mathbf{h}}|}$ on the objective

$$\text{CE}\left(\text{GetLogits}(\mathcal{N}_{\Pi_{\mathbf{h}} \leftarrow \mathbf{m} \circ \text{Get}(\mathcal{N}(c), \Pi_{\mathbf{h}})}(b)), \text{GetLogits}(\mathcal{H}_{X \leftarrow \text{Get}(\mathcal{H}(c), X)}(b))\right) \quad (8)$$

Principal Component Analysis. Principal Component Analysis (PCA) serves as an unsupervised dimensionality reduction technique (Tigges et al., 2023; Marks & Tegmark, 2024). For a vector set $(\mathcal{V} \subset \mathbb{R}^n)$ where $(|\mathcal{V}| > n)$, PCA determines orthogonal unit vectors $[\mathbf{p}_1 \dots \mathbf{p}_n]$. We employ the principal components’ orthogonal matrix as featurizer $\mathcal{F}$, mapping neurons into a more interpretable lower-dimensional space. Given PCA’s unsupervised nature, which doesn’t inherently specify component information, we use differential binary masking to select principal components that best abstracted by a causal variable.

Sparse Autoencoders. Sparse Autoencoders (SAE) employ an autoencoder architecture to transform neural activations into a sparse, higher-dimensional feature space before reconstruction (Bricken et al., 2023; Huben et al., 2024). Our implementation utilizes the GemmaScope (Lieberum et al., 2024) and LlamaScope (He et al., 2024) SAE collections. A key consideration is that featurizer invertibility requires inclusion of the SAE reconstruction loss. Consequently, all SAE feature interventions incorporate the base input’s reconstruction error. As with PCA, sparse autoencoders produce unsupervised features without inherent interpretability. We address this by implementing the previously described differential binary masking approach on SAE features (Chaudhary & Geiger, 2024).

F.3. Distributed Alignment Search

Distributed Alignment Search. Distributed Alignment Search (DAS) (Geiger et al., 2024b) operates as a supervised featurization technique that identifies a linear subspace within the model’s representation space. The method utilizes an orthogonal matrix Q of size $n \times n$, written as $Q = [\mathbf{u}_1 \dots \mathbf{u}_n]$. This transformation matrix converts the original representation into a new coordinate system through $\mathcal{F}(\mathbf{h}) = Q^\top \mathbf{h}$. The feature subset $\Pi_{\mathbf{h}}$ is extracted from the first k dimensions of this transformed space, where k serves as an adjustable hyperparameter. The optimization of matrix Q minimizes the following loss:

$$\mathcal{L} = \text{CE}(\mathcal{H}_{X \leftarrow \text{Get}(\mathcal{H}(c), X)}(b), \mathcal{N}_{\Pi_{\mathbf{h}}^X \leftarrow \text{Get}(\mathcal{N}(c), \Pi_{\mathbf{h}}^X)}(b))$$

To manage computational efficiency, rather than computing the complete matrix Q , we learn only the k orthogonal vectors that constitute feature $\Pi_{\mathbf{h}}^X$. Our implementation utilizes the pyvene library (Wu et al., 2024), training the featurizer on base-counterfactual pairs with interchange interventions.

F.4. Hyperparameters

Learning rate and regularization The learning rate used across models and tasks was 0.01, except for IOI which we used learning rate of 1.0. No regularization loss terms were used.

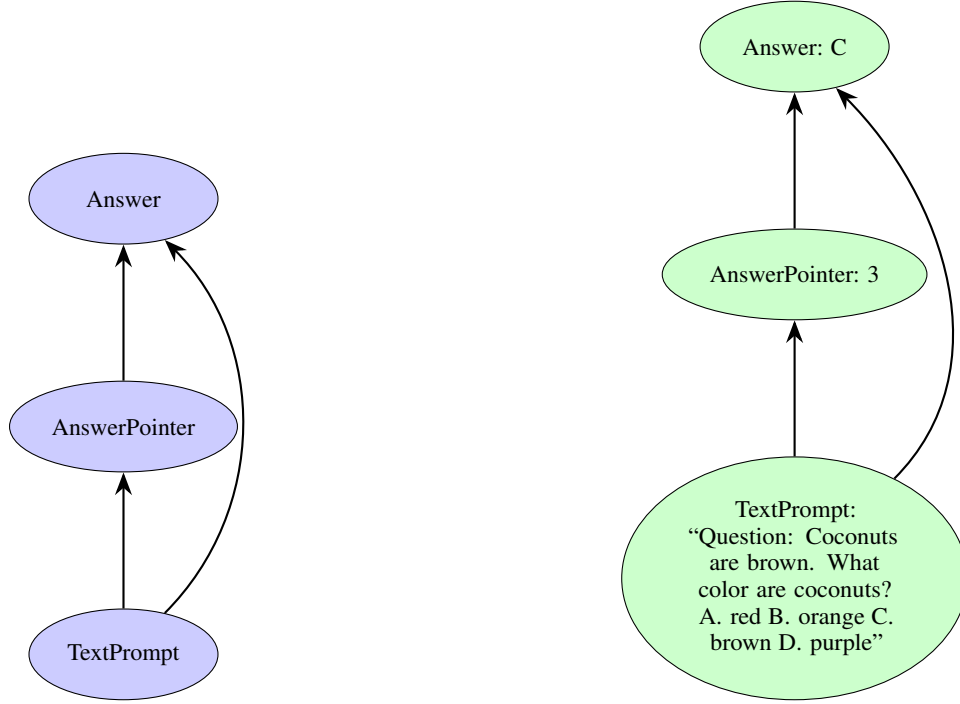
Epochs and batch size. For RAVEL, we train for one epoch of $\approx 30k$ examples with a batch size of 128 for Llama and 32 for Gemma. For MCQA, we train for 8 epochs on ≈ 300 examples with a batch size of 64. For ARC (easy), we train for 2 epochs of $\approx 9k$ examples and a batch size of 48 with Gemma and for 1 epoch with a batch size of 16 for Llama. For the two-digit addition task, we train for 1 epoch on $\approx 30k$ examples with a batch size of 256. For IOI, we train for one epoch on $\approx 30k$ examples.

DAS dimensionality. The dimensionality of DAS was set at 16 for the ordering ID X_{Order} and carry-the-one variable X_{Carry} . The DAS dimensionality for S_{Tok} and S_{Pos} are 32. The O_{Answer} variable in MCQA and ARC (Easy) has a DAS dimensionality of half the residual stream for their respective model, because token embeddings live in a higher dimensional space. The RAVEL task which had the dimensionality of an eighth of the residual stream, according to the experiments from (Huang et al., 2024a).

Masking parameters. For the masking methods, the temperature schedule used begins at 1.0 and approaches 0.01.

F.5. High-level Causal Models and Experimental Details for Each Task

F.5.1. MULTIPLE-CHOICE QUESTION ANSWERING



(a) General causal model for multiple choice question answering

(b) Specific example of the causal process

Figure 7. Causal model for multiple choice question answering. The model operates through a two-step mechanism: first, the TextPrompt is processed to generate an AnswerPointer that identifies the position of the correct answer in the options list. Second, this AnswerPointer is used to extract the corresponding letter label (A, B, C, or D) as the final Answer. This mechanism separates the reasoning process (identifying which option is correct) from the answer extraction (converting position to label).

We define the causal model for multiple-choice question answering, including one for the ARC dataset, in Figure 7. It comprises two variables, as illustrated in Figure 7a: 1) X_{Order} : It takes the text prompt as input and outputs a pointer that encodes the position of the correct label. 2) O_{Answer} : It receives the answer pointer as input and dereferences it by retrieving